

Input / Output & Pipes

Section 8 — Detailed Command Reference | nezuko homelab

8.1 Standard Streams & File Descriptors

What are Standard Streams?

Every Linux program automatically gets three open communication channels when it starts — called **standard streams**. Think of them like three pipes permanently attached to every program: one for receiving input, one for sending normal output, and one for sending errors. By default all three connect to your terminal, but you can redirect any of them to files, other commands, or nowhere at all.

Stream	Name	File Descriptor	Default source/destination	Purpose
stdin	Standard Input	0	Keyboard	Where the program reads input from
stdout	Standard Output	1	Screen	Where normal output is written
stderr	Standard Error	2	Screen	Where error messages are written

Why do stderr and stdout matter separately?

On screen stdout and stderr look identical — both print text. The difference only becomes visible when you redirect. If you redirect stdout to a file, errors still print to screen. If you redirect stderr to a file, normal output still prints to screen. This lets you separate useful output from error noise — essential in scripting.

Seeing streams in action:

```
cat # Reads from stdin (keyboard), writes to stdout (screen)
# Type anything and press Enter – it echoes back
# Press Ctrl+D to signal end of input

ls /nonexistent # stdout is empty, error goes to stderr
ls /etc /nonexistent # stdout = /etc listing, stderr = error message
# Both appear on screen but are separate streams
```

8.2 Redirection Operators

What is Redirection?

Redirection changes where a stream goes — instead of keyboard or screen, you point it at a file, another stream, or /dev/null. The shell sets up redirection before the command runs, so the program itself never knows the difference.

All redirection operators at a glance:

Operator	What it does	Overwrites existing file?
>	Redirect stdout to a file	Yes — destroys existing content
>>	Append stdout to a file	No — adds to end of file
<	Read stdin from a file instead of keyboard	N/A
2>	Redirect stderr to a file	Yes
2>>	Append stderr to a file	No
&>	Redirect both stdout AND stderr to a file	Yes
&>>	Append both stdout AND stderr to a file	No
2>&1	Redirect stderr into wherever stdout is going	Depends on stdout target

Examples on nezuko:

```
# Redirect stdout to file (overwrite):
ls /etc > listing.txt # Save /etc listing to file
ls /home/scottp > home_listing.txt # Save your home dir listing

# Append stdout to file:
echo 'Line 1' > notes.txt # Create file with Line 1
echo 'Line 2' >> notes.txt # Add Line 2 — does not overwrite
cat notes.txt # Shows both lines

# Redirect stdin from file:
wc -l < /etc/passwd # Count lines in passwd without cat

# Redirect stderr only:
ls /nonexistent 2> errors.txt # Error saved to file, nothing on screen
ls /etc /nonexistent 2> errors.txt # /etc prints to screen, error saved

# Redirect both to same file:
ls /etc /nonexistent &> all.txt # Everything goes to file

# Redirect to separate files:
ls /etc /nonexistent > out.txt 2> err.txt # Split streams cleanly

# Discard all output:
sudo apt update &> /dev/null # Run silently — no output at all
```

■ **Never redirect output to the same file you are reading from in the same command: `sort names.txt > names.txt` ← THIS WIPES THE FILE EMPTY** The shell opens the file for writing before the command reads it. Always use a different filename or use the `-o` flag (`sort -o names.txt names.txt`).

Important File: /dev/null

File path:

/dev/null — the null device, commonly called the black hole of Linux.

What it does:

Anything written to /dev/null is immediately discarded — it never appears anywhere. Reading from /dev/null returns nothing (immediate EOF). It is not a real file on disk — it is a special device provided by the kernel.

Permissions:

```
ls -la /dev/null
# crw-rw-rw- 1 root root 1, 3 ... /dev/null
# c = character device (not a regular file)
# rw-rw-rw- = everyone can read and write to it
```

Common uses:

Command	What it achieves
command > /dev/null	Discard stdout — suppress normal output
command 2> /dev/null	Discard stderr — suppress error messages only
command &> /dev/null	Discard everything — completely silent
cat /dev/null > file.txt	Empty a file without deleting it
sudo apt update 2>/dev/null # Hide any apt warnings on nezuko find / -name '*.log' 2>/dev/null # Search without permission denied errors cat /dev/null > /home/scottp/test.txt # Empty a file cleanly	

8.3 Pipes |

What is a Pipe?

The pipe operator `|` connects the stdout of one command directly to the stdin of the next — no temporary file needed. Data flows from left to right through the pipeline in real time. This is the Unix philosophy: small tools that do one thing well, combined to do powerful things.

How a pipeline works:

```
ls /etc | head -5
# Step 1: ls /etc produces a list of files → stdout
# Step 2: | takes that stdout and feeds it to head as stdin
# Step 3: head -5 reads 5 lines and prints them
# Result: first 5 entries of /etc – no temp file created
```

Chaining multiple pipes:

```
cat /etc/passwd | grep bash | wc -l
# Step 1: cat reads /etc/passwd → stdout
# Step 2: grep filters lines containing 'bash'
# Step 3: wc -l counts the remaining lines
# Result: number of users whose shell is bash

# Practical examples on nezuko:
history | grep ssh # Find SSH commands in your history
ps aux | grep docker # Find Docker processes
ls /home/scottp | wc -l # Count files in your home directory
du -h /var/log/* 2>/dev/null | sort -h | tail -10 # Largest log files
journalctl -u ssh | grep Accepted # All successful SSH logins
```

■ Pipes only connect stdout to stdin. stderr is NOT passed through a pipe by default. To also pipe stderr, merge it first: `command 2>&1 | nextcommand`

8.4 Filter Commands — Designed for Pipelines

Filter commands read from stdin, process the data, and write to stdout — making them perfect for use in pipelines. They can also work on files directly. These are the building blocks of powerful one-liners.

sort

Sort lines of text alphabetically or numerically

What it does:

sort reads lines from a file or stdin and outputs them in sorted order. Default is alphabetical ascending. Does not modify the original file — output goes to stdout unless redirected.

Common options:

Option	What it does
sort file	Sort file alphabetically A-Z
sort -r file	Reverse sort Z-A
sort -n file	Sort numerically (10 comes after 9, not between 1 and 2)
sort -h file	Sort by human readable sizes (K, M, G) — use with du output
sort -u file	Sort and remove duplicates in one step
sort -k2 file	Sort by column 2 (space separated)
sort -t: -k3 file	Sort by column 3 using : as delimiter
sort -o file file	Sort file and write back to same file safely (avoids overwrite trap)

```
sort names.txt # Sort names.txt alphabetically
sort -r names.txt # Reverse sort
sort -u names.txt # Sort and deduplicate in one go
sort -o names.txt names.txt # Sort names.txt and overwrite safely

# Useful on nezuko:
ps aux --sort=-%mem | head -5 # Top 5 memory users (ps has built-in sort)
du -h /var/log/* 2>/dev/null | sort -h | tail -5 # Largest log files
cat /etc/passwd | sort -t: -k3 -n # Sort passwd by UID numerically
```

uniq

Remove or count duplicate adjacent lines

What it does:

uniq removes duplicate lines — but only if they are adjacent (next to each other). This is why uniq is almost always used after sort. Sort groups duplicates together, then uniq removes them.

Common options:

Option	What it does
uniq file	Remove adjacent duplicate lines
uniq -c file	Prefix each line with count of occurrences
uniq -d file	Print only lines that ARE duplicated
uniq -u file	Print only lines that are NOT duplicated (unique only)
uniq -i file	Case insensitive comparison

```
sort names.txt | uniq # Remove duplicates from names.txt
sort names.txt | uniq -c # Count occurrences of each name
sort names.txt | uniq -c | sort -rn # Most frequent names first

# On nezuko – find most common SSH source IPs:
journalctl -u ssh | grep Accepted | awk '{print $11}' | sort | uniq -c | sort -rn
```

■ **uniq only removes ADJACENT duplicates. Always sort first unless your data is already grouped. 'uniq names.txt' without sorting first will miss non-adjacent duplicates.**

cut

Extract specific columns or characters from lines

What it does:

cut extracts specific fields (columns) or character positions from each line. It is essential for working with structured text files like /etc/passwd where fields are separated by a delimiter like ':'.

Common options:

Option	What it does
-d:	Set delimiter to : (can be any character)
-f1	Extract field 1 (first column)
-f1,3	Extract fields 1 and 3
-f1-3	Extract fields 1 through 3
-c1-5	Extract characters 1 to 5 (position based, not delimiter based)
-c1	Extract just the first character of each line

```
cut -d: -f1 /etc/passwd # Extract all usernames (field 1)
cut -d: -f1,3 /etc/passwd # Extract username and UID
cut -d: -f1,6 /etc/passwd # Extract username and home directory
cut -d: -f7 /etc/passwd # Extract default shell for each user

echo 'hello world' | cut -c1-5 # Extract first 5 characters: hello
echo 'hello world' | cut -d' ' -f1 # Extract first word: hello

# Pipeline example on nezuko:
cut -d: -f1 /etc/passwd | sort # Sorted list of all usernames
cut -d: -f1,3 /etc/passwd | grep scottp # Find scottp's UID
```

tr

Translate, replace, or delete characters

What it does:

tr translates (replaces) or deletes individual characters from its input. It only reads from stdin — it cannot take a filename directly. It works character by character, not word by word.

Common options:

Option	What it does
tr 'set1' 'set2'	Replace each character in set1 with corresponding character in set2
tr -d 'set'	Delete all characters in set
tr -s 'set'	Squeeze repeated characters in set into one
tr 'a-z' 'A-Z'	Convert lowercase to uppercase
tr 'A-Z' 'a-z'	Convert uppercase to lowercase
tr ' ' '\n'	Replace spaces with newlines (one word per line)
tr -d '0-9'	Delete all digits
tr -d '\n'	Remove all newlines (join lines together)

```
echo 'HELLO' | tr 'A-Z' 'a-z' # Convert to lowercase: hello
echo 'hello' | tr 'a-z' 'A-Z' # Convert to uppercase: HELLO
echo 'hello world' | tr ' ' '_' # Replace spaces with underscores: hello_world
echo 'hello world' | tr ' ' '\n' # One word per line
echo 'abc123def' | tr -d '0-9' # Remove digits: abcdef

# Practical on nezuko:
cat /etc/passwd | tr ':' '\t' # Replace colons with tabs for readability
echo $PATH | tr ':' '\n' # Show each PATH entry on its own line
```


tee

Write to a file AND pass output on down the pipeline

What it does:

tee is named after the T-junction in plumbing. It receives stdin, writes it to a file, AND passes it on to stdout simultaneously. Without tee you have to choose between saving to a file OR piping to the next command. tee lets you do both at the same time.

Common options:

Option	What it does
tee file	Write to file (overwrite) AND pass to stdout
tee -a file	Append to file AND pass to stdout
tee file1 file2	Write to multiple files AND pass to stdout

Position of tee matters:

Where you place tee in the pipeline determines what gets saved. Before a filter saves everything. After a filter saves only the filtered result.

```
ls /etc | tee listing.txt # Show on screen AND save full list
ls /etc | tee listing.txt | head -5 # Save full list, show only first 5

# tee BEFORE grep – saves everything, shows only matches:
ls /etc | tee listing.txt | grep 'conf'

# tee AFTER grep – saves only matches, shows only matches:
ls /etc | grep 'conf' | tee conf_files.txt

# Pipeline challenge solution on nezuko:
cut -d: -f1 /etc/passwd | sort | tee users.txt | wc -l
# Saves sorted usernames to users.txt AND counts them

# Append mode – add to existing log:
ps aux | grep jellyfin | tee -a jellyfin_checks.txt
```

xargs

Convert stdin into command arguments

What it does:

Some commands cannot read from stdin — they only accept arguments. xargs bridges this gap by reading stdin and converting it into arguments for another command. Think of it as turning a list of items into individual inputs to a command.

Common options:

Option	What it does
xargs command	Pass all stdin as arguments to command in one go
xargs -n 1 command	Pass one argument at a time — runs command once per item
xargs -n 2 command	Pass two arguments at a time
xargs -l {} command {}	Use {} as placeholder for each input item
xargs -p command	Prompt before each execution (interactive/safe mode)

```
cat names.txt | xargs echo 'Hello:' # All names as one echo argument
cat names.txt | xargs -n1 echo 'Hi:' # One echo per name

# Create a file for each unique name:
sort names.txt | uniq | xargs touch # Creates Alice, Bob, Charlie files

# Practical on nezuko:
ls *.txt | xargs wc -l # Count lines in all txt files
cat userlist.txt | xargs -n1 id # Look up id info for each username
```

■ ADDITIONAL: xargs with find — one of the most powerful combinations in Linux

Added because: xargs is most commonly used with find in real sysadmin work.
find produces a list of files that xargs passes to another command as arguments.
This is far more efficient than running a command for each file individually.

```
find /home/scottp -name '*.txt' | xargs wc -l # Count lines in all txt files
find /var/log -name '*.log' | xargs grep 'error' # Search all logs for errors
find /tmp -name 'test*' | xargs rm # Delete all test files in /tmp
```

■ If filenames contain spaces, use: `find ... -print0 | xargs -0` command
The `-0` flag handles null-terminated filenames safely.

Command Chaining Operators

&& || ;

Control how multiple commands run based on success or failure

What they do:

These operators let you run multiple commands in sequence with control over what happens when a command succeeds or fails. Every command returns an exit code: 0 means success, anything else means failure.

Operator	Meaning	When next command runs
&&	AND — run next only if previous succeeded	Only if exit code = 0 (success)
	OR — run next only if previous failed	Only if exit code ≠ 0 (failure)
;	Separator — always run next	Always, regardless of exit code

&& examples:

```
sudo apt update && sudo apt upgrade # Only upgrade if update succeeded
cd /home/scott && ls # Only ls if cd worked
ping -c1 google.com && echo 'Online' # Only print Online if ping worked
sleep 10 && echo 'Done' # Print Done after 10 second wait
```

|| examples:

```
ping -c1 google.com || echo 'Offline' # Print Offline if ping failed
mkdir /tmp/test || echo 'Already exists' # Warn if dir exists
```

; examples:

```
echo 'Starting'; sleep 2; echo 'Done' # Run all three regardless
sudo apt update; sudo apt upgrade # Run both, second runs even if first fails
```

Concept: TTY, Interactive vs Non-Interactive

What is a TTY?

TTY stands for TeleTYpewriter — a historical name from when physical terminals were actual typewriter machines. In modern Linux, a TTY is a terminal interface. There are two types:

Type	Name	Example	TTY column in ps
tty	Physical/virtual console	Direct keyboard on nezuko	tty1, tty2...
pts	Pseudo terminal slave	SSH session, terminal emulator	pts/0, pts/1...

Interactive vs Non-Interactive:

An **interactive** shell is one attached to a terminal where a human is typing. A **non-interactive** shell has no terminal — it runs scripts or cron jobs. This distinction matters because some commands and features only work in interactive shells (like command history, tab completion, and colour output).

Type	Has TTY?	Examples	Behaviour
Interactive	Yes — pts or tty	SSH session, terminal window	Colours, history, prompts work
Non-interactive	No — shows ?	Cron jobs, scripts, pipes	No colours, no prompts, minimal environment

```
tty # Show which TTY your current session is on
who # See all logged in sessions and their TTYS
w # Sessions plus what each is currently running
ps aux | grep pts # Find all SSH/terminal sessions
```

Challenge — Analyse /etc/passwd with a Pipeline

Full solution with explanation:

```
# Step 1 – Extract just usernames (field 1, colon delimited):
cut -d: -f1 /etc/passwd

# Step 2 – Add sort:
cut -d: -f1 /etc/passwd | sort

# Step 3 – Count them:
cut -d: -f1 /etc/passwd | sort | wc -l

# Step 4 – Save AND display simultaneously:
cut -d: -f1 /etc/passwd | sort | tee users.txt

# Full pipeline – save sorted list AND count in one command:
cut -d: -f1 /etc/passwd | sort | tee users.txt | wc -l
# users.txt = sorted username list
# Terminal = count of usernames

# Verify:
cat users.txt # Confirm sorted list saved correctly
```

Quick Reference — All Operators and Commands

Redirection operators:

Operator	What it does
>	Redirect stdout to file (overwrite)
>>	Append stdout to file
<	Read stdin from file
2>	Redirect stderr to file
2>>	Append stderr to file
&>	Redirect both stdout and stderr to file
&>>	Append both stdout and stderr to file
2>&1	Merge stderr into stdout
	Pipe stdout to next command's stdin
&&	Run next command only if previous succeeded
	Run next command only if previous failed
;	Run next command regardless of previous result

Filter commands:

Command	What it does
sort file	Sort lines alphabetically
sort -r file	Reverse sort
sort -n file	Numeric sort
sort -h file	Human size sort (K M G)
sort -u file	Sort and deduplicate
uniq	Remove adjacent duplicate lines (sort first!)
uniq -c	Count occurrences of each line
uniq -d	Show only duplicated lines
cut -d: -f1	Extract field 1 using : as delimiter
cut -c1-5	Extract characters 1-5
tr 'A-Z' 'a-z'	Convert to lowercase
tr -d '0-9'	Delete digits
tr ' ' '\n'	Replace spaces with newlines
tee file	Write to file AND pass on to stdout
tee -a file	Append to file AND pass on to stdout
xargs command	Convert stdin to command arguments
xargs -n1 command	One argument at a time

■ Linux+ Exam Tips: Know all three streams — stdin(0) stdout(1) stderr(2). Know > overwrites, >> appends. Know 2> is stderr, &> is both. Know that /dev/null discards output. Know | only pipes stdout not stderr — use 2>&1 to include stderr in a pipe. Know uniq needs sort first. Know cut -d sets delimiter and -f sets field number. Know && runs next only on success, || runs next only on failure, ; always runs next.